Spring Boot Essentials

A short practical guide to building REST services with Spring Boot

1. Introduction to Spring Boot

Spring Boot is a framework built on top of the Spring Framework that makes it faster and easier to create production-ready Java applications. Its guiding principle is convention over configuration: instead of asking the developer to write long configuration files, Spring Boot provides sensible defaults and lets the developer override only what needs to change.

Three ideas explain most of Spring Boot. The first is starter dependencies: a starter such as spring-boot-starter-web is a single dependency that pulls in a coherent set of libraries for a given purpose, with compatible versions. The second is auto-configuration: Spring Boot inspects the classpath, the beans already defined and the configuration properties, and configures the application accordingly. The third is the embedded server: an application is packaged as a standalone executable jar that contains its own web server, which is Tomcat by default, so no external application server has to be installed.

Spring Boot also ships production-oriented features out of the box, including health checks, metrics, externalized configuration and structured logging. These are mostly provided by the Actuator module described later in this guide.

It is important to understand what Spring Boot is not. It does not generate code, and it does not require XML configuration. It is not a replacement for the Spring Framework either: Spring Boot applications still use Spring concepts such as dependency injection, the application context and Spring MVC. Spring Boot simply removes most of the setup work around them.

This guide walks through the lifecycle of a typical Spring Boot REST service: creating the project, understanding its structure, writing controllers, accessing a database, validating input, handling errors, testing, monitoring, securing, packaging and containerizing the application.

2. Creating a Project

The easiest way to start a new Spring Boot application is Spring Initializr, available at start.spring.io. It is a web tool that generates a ready-to-run project skeleton. The developer chooses the build tool (Maven or Gradle), the language (Java, Kotlin or Groovy), the Spring Boot version, the project metadata (group, artifact and package name), the Java version and the dependencies.

Common dependencies for a REST API are Spring Web for controllers, Spring Data JPA for database access, Validation for input checks, Spring Boot Actuator for monitoring and Spring Security for authentication. Additional choices include a database driver such as PostgreSQL or MySQL, and H2 for an in-memory database during development.

The generated archive contains the build file (pom.xml for Maven or build.gradle for Gradle), the main application class, a file named application.properties inside src/main/resources, and a simple test class inside src/test/java. The Maven wrapper or Gradle wrapper is included, which means that no local Maven or Gradle installation is required.

To run the application during development, use the wrapper command mvnw spring-boot:run for Maven or gradlew bootRun for Gradle. The application starts the embedded Tomcat server and, unless configured otherwise, listens on port 8080. The same project can also be created directly from an IDE such as IntelliJ IDEA or Visual Studio Code, which both integrate with Spring Initializr.

3. Application Structure and the SpringBootApplication Annotation

Every Spring Boot application has a main class annotated with @SpringBootApplication. The main method of that class calls SpringApplication.run, which creates the application context, applies auto-configuration, scans for components and starts the embedded web server.

The @SpringBootApplication annotation is a convenience annotation that combines three other annotations. @Configuration allows the class to define beans. @EnableAutoConfiguration turns on Spring Boot's auto-configuration mechanism. @ComponentScan tells Spring to scan the package of the annotated class, and all of its sub-packages, for components such as controllers, services and repositories.

Because component scanning starts from the package of the main class, the main class should be placed in the root package of the application. If a controller or service lives in a package outside of this root, Spring will not find it and the application will fail with errors such as a missing bean or a controller that returns 404.

A typical project is organized in layers. The controller package contains the web layer that exposes HTTP endpoints. The service package contains the business logic. The repository package contains the interfaces used to access the database. The model or entity package contains the persistent classes, and the dto package contains the objects exchanged with clients. The config package contains configuration classes such as security settings. Some teams prefer to package by feature instead of by layer, grouping everything related to one business concept together. Both approaches work, as long as the team stays consistent.

4. Auto-Configuration

Auto-configuration is the mechanism that lets Spring Boot configure an application automatically, based on what is present on the classpath, which beans are already defined and which properties are set. For example, when spring-boot-starter-web is on the classpath, Spring Boot configures Spring MVC and an embedded Tomcat server. When an H2 database driver is on the classpath and no DataSource has been defined, Spring Boot configures an in-memory database automatically.

Auto-configuration classes rely on conditional annotations. @ConditionalOnClass activates a configuration only when a given class is available. @ConditionalOnMissingBean activates it only when the developer has not defined a bean of the same type. @ConditionalOnProperty activates it depending on the value of a property. This design means that auto-configuration always backs off when the developer defines a custom bean: the developer's configuration takes priority over the default one.

When it is unclear why a particular bean was or was not configured, the developer can start the application with the debug flag, either by passing --debug on the command line or by setting debug=true in the properties file. Spring Boot then prints a conditions evaluation report that lists which auto-configurations matched and which did not, together with the reason.

A specific auto-configuration can be disabled with the exclude attribute of the @SpringBootApplication annotation, or with the property spring.autoconfigure.exclude. This is useful, for example, to disable the default security configuration or the automatic DataSource configuration in a project that does not use a database yet.

5. Dependency Injection and Beans

At the heart of Spring is the application context, a container that creates, configures and manages objects called beans. Instead of creating its own dependencies with the new keyword, a class declares what it needs and Spring provides it. This is called dependency injection, and it makes code easier to test and to change.

Beans are registered in two main ways. The first is with stereotype annotations placed on classes: @Component is the generic one, @Service marks business logic, @Repository marks data access classes, and @Controller and @RestController mark web layer classes. These classes are discovered by component scanning. The second way is with @Bean methods declared inside a class annotated with @Configuration. This is useful for objects from third-party libraries that cannot be annotated, such as a password encoder or an HTTP client.

Constructor injection is the recommended style. The dependencies are declared as constructor parameters, ideally as final fields. This makes the class immutable, makes it clear what the class needs, and makes unit testing simple because the dependencies can be passed manually. When a class has a single constructor, Spring uses it automatically and the @Autowired annotation is not needed.

By default every bean is a singleton, meaning that one instance is shared across the application. Other scopes exist, such as prototype, request and session. Circular dependencies, where two beans depend on each other, should be avoided: since Spring Boot 2.6 they are prohibited by default and the application fails at startup with an explicit error. The usual fix is to refactor the design, for example by extracting the shared logic into a third class.

6. Building REST Controllers

A REST controller is a class annotated with @RestController, which combines @Controller and @ResponseBody. The return value of each method is written directly to the HTTP response body, and Spring Boot uses the Jackson library to convert Java objects to JSON automatically.

HTTP methods are mapped with dedicated annotations: @GetMapping for reading, @PostMapping for creating, @PutMapping for replacing, @PatchMapping for partial updates and @DeleteMapping for deleting. A @RequestMapping annotation placed on the class defines the base path shared by all methods, for example /api/products.

Request data is bound to method parameters with annotations. @PathVariable extracts a value from the URL path, such as the identifier in /api/products/42. @RequestParam reads a query parameter, such as a page number or a search term. @RequestBody converts the JSON body of the request into a Java object.

A method can return a ResponseEntity to control the status code and the headers of the response. Good REST practice is to return 200 for a successful read, 201 Created with a Location header after a creation, 204 No Content after a deletion, 400 for invalid input and 404 when a resource does not exist. Controllers should exchange data transfer objects (DTOs) with clients rather than exposing database entities directly, because entities may contain sensitive fields and tie the public API to the database schema. Controllers should stay thin and delegate business logic to services.

7. Configuration Properties and Profiles

Spring Boot externalizes configuration so that the same application can run in different environments without being rebuilt. Properties are normally written in application.properties or application.yml inside src/main/resources. Common examples are server.port to change the HTTP port, spring.datasource.url for the database connection, and logging.level.root to set the log level.

There are two ways to read properties in code. The @Value annotation injects a single value into a field or a constructor parameter. The @ConfigurationProperties annotation binds a whole group of properties with a common prefix to a typed class, which is cleaner for larger sets of settings and supports validation. A properties class is registered with @EnableConfigurationProperties or with @ConfigurationPropertiesScan.

Profiles let the developer define different configuration for different environments. A file named application-dev.properties is loaded only when the dev profile is active, and application-prod.properties only when the prod profile is active. A profile is activated with the property spring.profiles.active, or with the environment variable SPRING_PROFILES_ACTIVE, for example when starting a container.

Spring Boot applies a clear order of precedence. Environment variables override the properties files, and command-line arguments override environment variables. Secrets such as database passwords, API keys and signing keys must never be committed to the repository. They should be provided through environment variables or a secrets manager, and the properties file should reference them with placeholders such as the syntax dollar sign followed by curly braces around the variable name.

8. Data Access with Spring Data JPA

The starter spring-boot-starter-data-jpa brings Hibernate as the JPA implementation together with Spring Data JPA. A persistent class is annotated with @Entity, its primary key with @Id, and an automatically generated key with @GeneratedValue. Relationships between entities are declared with annotations such as @OneToMany, @ManyToOne and @ManyToMany.

A repository is an interface that extends JpaRepository, parameterized with the entity type and the type of its identifier. Spring Data generates the implementation at runtime, so methods such as save, findById, findAll and deleteById are available without writing any code. The developer can also declare derived query methods whose name describes the query, for example findByEmail or findByStatusOrderByCreatedAtDesc. For more complex queries, the @Query annotation accepts a JPQL or native SQL statement.

The property spring.jpa.hibernate.ddl-auto controls how Hibernate manages the schema. The values include none, validate, update, create and create-drop. The update and create modes are convenient during development, but in production the schema should be managed with a migration tool such as Flyway or Liquibase, which version the database changes in scripts. The property spring.jpa.show-sql prints the generated SQL and is helpful for debugging.

Transactions are declared with the @Transactional annotation, usually on service methods. A common performance trap is the N+1 query problem, where loading a list of entities triggers one additional query for each related entity. It can be solved with fetch joins, entity graphs or by projecting the data directly into DTOs. Another common error is the LazyInitializationException, which occurs when a lazy relationship is accessed after the transaction has closed.

9. Input Validation

Validation protects the application from invalid or malicious input. Spring Boot supports the Jakarta Bean Validation standard through the starter spring-boot-starter-validation, which uses Hibernate Validator as its implementation.

Constraints are declared as annotations on the fields of a DTO. The most common are @NotNull to forbid null values, @NotBlank to forbid empty or whitespace-only text, @Size to limit length, @Email to check an email format, @Min and @Max for numeric bounds, @Positive for positive numbers and @Pattern for regular expressions. Each annotation accepts a message attribute to customize the error text.

To trigger validation, the @Valid annotation is added to the @RequestBody parameter of a controller method. When the data violates a constraint, Spring throws a MethodArgumentNotValidException and the response has the status 400 Bad Request. Parameters that are not part of the body, such as path variables and query parameters, are validated by placing @Validated on the controller class and constraint annotations directly on the method parameters.

When the standard constraints are not enough, the developer can create a custom constraint by writing an annotation and a class implementing ConstraintValidator, for example to check that a phone number matches a national format. Validation should be performed at the boundary of the application, in the controller layer, and complemented by database constraints as a last line of defense.

10. Exception Handling

Without any customization, an unhandled exception produces a generic error response. A good API returns consistent, informative error messages with the right status code. Spring offers a centralized way to do this through a class annotated with @RestControllerAdvice, which contains methods annotated with @ExceptionHandler. Each method handles one type of exception and returns the response that the client should receive.

A typical design defines custom exceptions that express business errors, such as ResourceNotFoundException for a missing record or DuplicateEmailException for a conflict. The handler maps them to the appropriate HTTP statuses: 404 Not Found, 409 Conflict, 400 Bad Request, and so on. A final handler for the generic Exception type returns a 500 Internal Server Error with a neutral message, while the real exception is written to the logs.

The error body should have a stable structure so that clients can parse it. Recent versions of Spring support the ProblemDetail class, which implements the standard problem details format for HTTP APIs, with fields such as type, title, status and detail. It can be enabled for built-in exceptions with the property spring.mvc.problemdetails.enabled.

Error responses must never leak internal details such as stack traces, SQL statements or file paths. By default Spring Boot does not include the stack trace in error responses, controlled by the property server.error.include-stacktrace, and it should stay that way in production. Detailed information belongs in the logs, correlated with a request identifier.

11. Testing Spring Boot Applications

The starter spring-boot-starter-test provides the common testing tools: JUnit 5 as the test framework, Mockito for mocks, AssertJ for readable assertions, and the Spring Test utilities, including MockMvc.

The @SpringBootTest annotation starts the full application context and is suited to integration tests, but it is relatively slow. For faster feedback, Spring Boot offers slice tests that load only part of the application. The @WebMvcTest annotation loads only the web layer for testing controllers with MockMvc, and the services that the controller depends on are replaced by mocks using @MockBean, or @MockitoBean in recent versions. The @DataJpaTest annotation loads only the JPA components and uses an in-memory database by default, which is ideal for testing repositories and custom queries.

Pure unit tests do not need Spring at all. A service that receives its dependencies through the constructor can be tested with plain JUnit and Mockito, which makes these tests the fastest and the most numerous in a healthy test pyramid.

To test against a real database rather than an in-memory one, the Testcontainers library starts a disposable Docker container, for example PostgreSQL, during the tests. This avoids surprises caused by differences between H2 and the production database. Tests should run automatically in the continuous integration pipeline on every push, and tools such as JaCoCo measure code coverage while SonarQube analyzes the quality of the code.

12. Monitoring with Spring Boot Actuator

Spring Boot Actuator adds production-ready monitoring endpoints to an application. It is enabled by adding the starter spring-boot-starter-actuator. The endpoints are exposed under the path /actuator and include health, info, metrics, env, loggers, beans and mappings.

By default only the health endpoint is exposed over HTTP. Other endpoints are made available with the property management.endpoints.web.exposure.include, which accepts a list of endpoint names. Because endpoints such as env and beans can reveal sensitive information, they must be secured or kept on an internal network and never exposed publicly without protection.

The health endpoint reports whether the application and its dependencies, such as the database, are working. It is widely used by orchestrators: Kubernetes liveness and readiness probes can call the health endpoint to decide when to restart a container or when to send traffic to it. Custom health indicators can be added by implementing the HealthIndicator interface.

Actuator relies on Micrometer to collect metrics such as JVM memory usage, HTTP request counts and latencies, and database connection pool statistics. By adding the registry micrometer-registry-prometheus, the application exposes its metrics at the endpoint /actuator/prometheus in a format that Prometheus can scrape. The collected data can then be displayed in Grafana dashboards and used to trigger alerts.

13. Security Basics

Adding the starter spring-boot-starter-security secures the whole application immediately. By default all endpoints require authentication, and Spring Boot generates a random password for a user named user, which is printed in the logs at startup. This default is only meant for experiments and must be replaced in a real application.

Security is customized by declaring a bean of type SecurityFilterChain. In it the developer defines which paths are public, which paths require authentication, and which roles may access each path. Passwords must never be stored in clear text: the BCryptPasswordEncoder bean hashes passwords with a strong adaptive algorithm.

Stateless REST APIs usually authenticate requests with JSON Web Tokens (JWT) sent in the Authorization header as a bearer token. The starter spring-boot-starter-oauth2-resource-server validates the token signature and extracts the claims and roles. In larger systems the tokens are issued by a dedicated identity provider such as Keycloak, which centralizes authentication, user management and single sign-on, and the Spring Boot services only validate the tokens.

Method-level security is enabled with @EnableMethodSecurity, which makes annotations such as @PreAuthorize available, for example to restrict a method to users with the ADMIN role. CSRF protection is enabled by default because it protects browser sessions; it is commonly disabled for stateless APIs that do not rely on cookies. When a browser frontend on a different origin calls the API, CORS must be configured explicitly, otherwise the browser blocks the requests.

14. Packaging and Running

A Spring Boot application is packaged as an executable jar, often called a fat jar, that contains the compiled classes, all the dependencies and a special launcher. With Maven, the command mvnw clean package builds the jar in the target directory using the spring-boot-maven-plugin. The application is then started with the command java -jar followed by the path of the jar.

Properties can be overridden at startup without rebuilding the jar. For example, passing --server.port=9090 changes the port, and passing --spring.profiles.active=prod activates the production profile. Environment variables work the same way, which is the preferred method in containers and cloud platforms.

Spring Boot supports graceful shutdown, enabled with the property server.shutdown=graceful. When the application receives a termination signal, it stops accepting new requests and waits for the active requests to finish before exiting. This prevents errors during rolling deployments.

Logging uses Logback by default. The log level can be set globally with logging.level.root, or per package, for example logging.level.com.example.shop=DEBUG. In production, logs are usually written to the standard output so that the platform can collect them, and they may be formatted as JSON for easier searching in tools such as Elasticsearch or Loki. Packaging as a WAR file for deployment to an external application server is still possible, but it is rarely needed today.

15. Containerizing with Docker

Containerizing a Spring Boot application makes it portable and simple to deploy. A minimal Dockerfile starts from a Java runtime image such as eclipse-temurin, copies the jar into the image and defines the entry point as java -jar. A multi-stage build is better: the first stage uses a full JDK image to compile and package the application, and the second stage copies only the jar into a small runtime image, which reduces the size and the attack surface.

Spring Boot can also build an image without a Dockerfile. The command mvnw spring-boot:build-image uses Cloud Native Buildpacks to produce an optimized image with sensible defaults. Layered jars split the application into dependency layers and an application layer, so that rebuilding after a small code change reuses the cached dependency layers.

Good practices include running the container as a non-root user, passing configuration through environment variables, and never baking secrets into the image. The JVM is aware of container memory limits, but the memory settings should still be tested under realistic load. A Docker health check can call the Actuator health endpoint.

Docker Compose is convenient to run the application together with its dependencies, such as PostgreSQL, Redis or Keycloak, on a developer machine with a single command. In a continuous integration and delivery pipeline on GitLab CI or GitHub Actions, a typical sequence is to build the project, run the tests, analyze code quality with SonarQube, build the Docker image, scan it for vulnerabilities with Trivy, push it to a registry and deploy it, for example to Kubernetes.

16. Common Problems and Solutions

Port already in use: the error message says that port 8080 is already in use. Either stop the process that occupies the port, or change the port of the application with the property server.port.

Failed to configure a DataSource: this error appears when a database starter is on the classpath but the connection properties are missing. Set spring.datasource.url, spring.datasource.username and spring.datasource.password, or add an in-memory database such as H2 for development.

NoSuchBeanDefinitionException or a bean that cannot be found: the class is usually outside the packages scanned from the main class, or it lacks a stereotype annotation such as @Service or @Component. Move the class under the root package or annotate it correctly.

A 404 error on an endpoint: check the mapping annotations, the base path declared on the class, and that the controller is in a scanned package. A Whitelabel Error Page means that Spring Boot is running but no handler matches the requested URL.

LazyInitializationException: a lazily loaded relationship was accessed after the transaction ended. Load the data inside the transaction, use a fetch join, or map the entity to a DTO within the service method.

401 or 403 responses after adding Spring Security: all endpoints are protected by default. Declare a SecurityFilterChain bean that permits public paths, and check the roles required by protected paths.

CORS errors in the browser: the frontend runs on a different origin than the API. Configure the allowed origins, methods and headers with a CorsConfigurationSource bean or the @CrossOrigin annotation, and remember that the preflight OPTIONS request must also be allowed.